Anomaly Detection — Claude Implementation Prompt

ANOMALY DETECTION — CLAUDE IMPLEMENTATION PROMPT

You are a Senior Data Scientist + AI Engineer working on an existing production-oriented project called:

AI Root Cause Analysis Platform

Your task is to IMPLEMENT ONLY the Anomaly Detection Feature.

Do not implement Root Cause Analysis, Ollama, AI agents, memory, RAG, or any other future feature.

1. EXISTING FEATURES — DO NOT REBUILD

The following features already exist and are working:
- Data ingestion
- Data profiling
- Schema validation
- KPI detection
- KPI selection
- KPI definition
- SQL Server integration
- Parquet-based dataset storage
- DuckDB analytical access

Do NOT rebuild, duplicate, or redesign these features unnecessarily.

Your task starts from:

```
Analysis Ready Dataset + KPI Definition
        ↓
ANOMALY DETECTION
```

Before writing code, inspect the existing repository and understand how these features currently work.

2. FEATURE OBJECTIVE

The purpose of this feature is to answer:

"Is this KPI behavior unusual compared with its historical behavior?"

Example:

Revenue
January $9.8M
February $10.1M
March $10.3M
April $10.0M
May $10.2M
June $10.1M
July $8.2M ← unusual

The system should identify whether July represents unusual KPI behavior and provide:
- anomaly status
- occurrence time
- actual value
- expected/baseline value
- deviation
- anomaly score
- severity
- direction
- detection method
- supporting historical evidence

3. IMPORTANT SCOPE

Implement ONLY:

```
KPI Time-Series Generation
        ↓
Historical Baseline
        ↓
Anomaly Detection
        ↓
Anomaly Score
        ↓
Severity
        ↓
Direction
        ↓
Structured Anomaly Result
        ↓
API
        ↓
Frontend Visualization
```

Do NOT implement:
- Root Cause Analysis
- Dimension contribution
- Root Cause Ranking
- RCA Tree
- Statistical hypothesis-testing framework
- Ollama
- LLM explanations
- RAG
- Memory
- Forecasting product
- Data ingestion
- Data profiling
- Schema validation
- KPI detection
- KPI selection
- SQL Server integration

4. START BY INSPECTING THE REPOSITORY

Inspect the existing backend:
- Dataset model/repository/service
- Dataset storage
- Parquet utilities
- DuckDB utilities
- KPI Definition schema/model/service
- Existing analytics modules
- API routes
- Pydantic schemas
- Error handling
- Tests

Inspect the frontend:
- Feature structure
- API client
- React Query/data fetching if used
- Dashboard components
- Plotly usage
- shadcn/ui components
- KPI UI

Do not assume the repository structure. Adapt to it and avoid duplicate utilities.

5. INPUT

The engine receives an existing Analysis Ready KPI definition, for example:

```
{
  "name": "Revenue",
  "column": "revenue",
  "aggregation": "SUM",
  "time_column": "date",
  "dimensions": ["region", "product"],
  "comparison": "previous_period"
}
```

Use the dataset, KPI column, aggregation, time column, and historical observations.

The engine must remain source-agnostic. It must not care whether the source was CSV, Excel, or SQL Server.

6. KPI TIME SERIES

Generate the KPI time series using the existing dataset and KPI definition.

Example:

Date Revenue
2026-01 9.8M
2026-02 10.1M
2026-03 10.3M
2026-04 10.0M
2026-05 10.2M
2026-06 10.1M
2026-07 8.2M

Respect the KPI aggregation:
- SUM
- AVG
- COUNT
- COUNT_DISTINCT
- MIN
- MAX
- MEDIAN

Do not hardcode SUM.

7. HISTORICAL BASELINE

Determine what normal KPI behavior looks like.

Consider:
- rolling mean
- rolling median
- rolling standard deviation
- MAD
- IQR
- exponentially weighted statistics
- seasonal baseline when sufficient history exists

Do not blindly implement every method. Prefer simple, explainable, robust methods.

Clearly document:
- selected baseline
- why it was selected
- minimum history
- calculation
- behavior at the beginning of the series

8. ANOMALY DETECTION

Implement a clear and explainable detection strategy.

Consider:
- Z-score
- Modified Z-score
- IQR
- MAD

- rolling deviation
- seasonal deviation

Use Scikit-learn only when it provides a clear benefit. Do not introduce complex ML unnecessarily.

Prioritize:
1. Explainability
2. Stability
3. Correctness
4. Performance

9. DETECTOR ARCHITECTURE

Allow additional detection methods later without rewriting the feature.

Conceptually:

```
AnomalyDetector
    ■■■ RobustZScoreDetector
    ■■■ IQRDetector
    ■■■ MADDetector
    ■■■ SeasonalDetector
```

Keep the abstraction small and adapt it to the existing architecture.

10. ANOMALY SCORE

Each evaluated observation should have an anomaly score.

Example:

Actual Revenue: $8.2M
Expected Revenue: $10.1M
Deviation: -$1.9M
Deviation: -18.8%
Anomaly Score: 3.7

The score must have a documented interpretation. Do not create an arbitrary score with no mathematical meaning.

11. SEVERITY

Use:
- NORMAL
- LOW
- MEDIUM
- HIGH
- CRITICAL

Use explicit named thresholds/configuration. Avoid scattered magic numbers.

12. DIRECTION

Identify:
- UPWARD
- DOWNWARD
- NONE

Example:
Expected $10M, Actual $8M → DOWNWARD
Expected $10M, Actual $13M → UPWARD

13. ANOMALY RESULT MODEL

Create a typed result using the project's Pydantic conventions.

Conceptually:

```
{
  "timestamp": "2026-07-01",
  "actual_value": 8200000,
```

```
  "expected_value": 10100000,
  "absolute_deviation": -1900000,
  "percentage_deviation": -18.8,
  "anomaly_score": 3.7,
  "severity": "HIGH",
  "direction": "DOWNWARD",
  "is_anomaly": true
}
```

Include relevant historical context where appropriate. Do not add meaningless fields.

14. NO ANOMALY

The engine must be able to return:

No anomaly detected.

Do not force every observation into an anomaly.

15. INSUFFICIENT HISTORY

Handle insufficient history safely. Do not produce a confident score with only a few observations.

Return an explicit state such as:

```
{
  "status": "INSUFFICIENT_HISTORY",
  "is_anomaly": false,
  "message": "Not enough historical observations to reliably detect anomalies."
}
```

Adapt naming to existing project conventions.

16. MISSING PERIODS

Do not automatically convert a missing period into zero unless business semantics explicitly say missing means zero.

Distinguish:
- missing observation
- zero value
- actual low value

17. ZERO AND NEGATIVE VALUES

Safely handle:
- zero KPI
- zero baseline
- negative KPI values
- negative historical values

Do not calculate percentage deviation using a zero denominator without defined behavior.

18. AGGREGATION AWARENESS

Operate on the resulting KPI time series, not raw rows.

Respect:
SUM, AVG, COUNT, COUNT_DISTINCT, MIN, MAX, MEDIAN.

This is primarily KPI-level time-series anomaly detection.

19. DUCKDB / PARQUET / PERFORMANCE

Reuse the existing Parquet + DuckDB infrastructure.

Prefer DuckDB for:
- time filtering
- aggregation
- date grouping
- KPI time-series generation

Avoid loading a large dataset entirely into Pandas just for GROUP BY operations.

Use Pandas/Polars only when needed by the anomaly algorithm.

20. BACKEND IMPLEMENTATION

Inspect the existing backend structure first.

```
If compatible, the implementation may conceptually contain:
backend/app/
    analytics/anomaly/
        models.py
        baseline.py
        detectors.py
        scoring.py
        classifier.py
    services/anomaly_service.py
    schemas/anomaly.py
    api/routes/anomaly.py
```

Do not blindly create these files. Reuse existing dataset access, DuckDB utilities, Pydantic patterns, service patterns, route patterns, and error handling.

21. API

Add an anomaly detection API following existing FastAPI conventions.

A possible endpoint is:

```
POST /api/anomaly/detect
```

The request should reference the existing dataset and KPI definition.

The response should contain:
- KPI information
- historical baseline
- anomaly observations
- severity
- direction
- method
- limitations

Use existing naming conventions if different. Do not create unnecessary endpoints.

22. FRONTEND

Implement only the frontend required for this feature.

Create an Anomaly Detection view showing:

KPI Summary
Revenue
Current: $8.2M
Expected: $10.1M
Deviation: -18.8%
Status: HIGH ANOMALY

Time-Series Chart:
Use Plotly to show:
- actual KPI
- expected/baseline
- anomalous points
- time axis
- useful hover information

Use existing Next.js, React, TypeScript, Tailwind, shadcn/ui, and Plotly.

Do not redesign unrelated screens.

23. TESTING

Implement unit tests for:
- KPI time-series generation
- baseline calculation

- anomaly detection
- anomaly score
- severity
- direction
- no anomaly
- insufficient history
- missing periods
- zero baseline
- negative values
- different aggregations

Implement an integration test:

```
Analysis Ready Dataset
        ↓
Anomaly API
        ↓
Anomaly Service
        ↓
Anomaly Detector
        ↓
Structured Result
```

24. GOLDEN DATASET

Use a deterministic dataset:

Month Revenue
Jan 1000
Feb 1020
Mar 980
Apr 1010
May 990
Jun 1005
Jul 600

Expected result:
July → DOWNWARD ANOMALY

The test must prove the algorithm discovers it. Do not hardcode July as an anomaly.

25. EDGE CASES

Test:
- empty dataset
- single observation
- very short history
- constant KPI
- no anomaly
- strong downward anomaly
- strong upward anomaly
- zero KPI
- zero baseline
- negative KPI
- missing periods
- missing time values
- duplicate periods
- very large values
- very small values
- NaN/null KPI values

Define expected behavior for each.

26. CODE QUALITY

Follow existing project conventions:
- Type hints

- Pydantic models where appropriate
- Separation of analytics and API
- Small testable functions
- No duplicated data-access logic
- No hardcoded business rules
- Named constants
- Clear error handling
- Useful logging
- No secrets in source code
- No unnecessary dependencies

Do not over-engineer.

27. DO NOT CHANGE PHASE 1

Do not refactor existing Phase 1 unless the anomaly feature genuinely requires a small integration change.

Prefer adapters or small extensions over rewriting existing services.

28. FUTURE RCA COMPATIBILITY

Future architecture:

```
Analysis Ready KPI
        ↓
Anomaly Detection
        ↓
Anomaly Result
        ↓
Root Cause Analysis
        ↓
RCA Result
        ↓
Ollama
        ↓
Business Explanation
```

Do NOT implement the future RCA Engine now.

Make the anomaly result structured and reusable.

29. DEFINITION OF DONE

The feature is complete when:
- Analysis Ready KPI can be submitted
- KPI time series is generated
- KPI aggregation is respected
- Historical baseline is calculated
- Anomalies are detected using an explainable method
- Anomaly score is generated
- Severity is calculated
- Direction is identified
- No-anomaly cases work
- Insufficient history is handled
- Missing periods are handled safely
- Zero/negative values are handled safely
- API returns a typed result
- Frontend visualizes the anomaly
- Plotly shows actual vs baseline
- Unit tests pass
- Integration tests pass
- Golden dataset test passes
- Existing Phase 1 functionality still works
- No RCA implementation was added
- No Ollama/LLM implementation was added

30. EXECUTION RULE

Follow this sequence:

1. Inspect the existing repository.

```
2. Identify dataset and KPI Definition interfaces.
3. Identify DuckDB/Parquet access.
4. Identify API/service/schema conventions.
5. Design the smallest implementation fitting the existing architecture.
6. Implement the backend anomaly engine.
7. Implement the API.
8. Implement the frontend visualization.
9. Add unit and integration tests.
10. Run the complete test suite.
11. Verify Phase 1 has not been broken.
```

At the end, report:
1. Files created
2. Files modified
3. Algorithms implemented
4. API endpoint added
5. Frontend components added
6. Tests added
7. Tests passed
8. Limitations or technical decisions

Do not implement anything outside the Anomaly Detection feature.